Asteroids: Data and Architecture Design

Skye Coppiëns, 6575439 Utku Özyurt, 7332424

October 14, 2026

1 Introduction and Rules

This document describes the design of our implementation of Asteroids, written in Haskell with Gloss. The game is played on a two-dimensional playing field, where the player controls a spaceship with the keyboard and shoots bullets to destroy different kinds of adversaries: asteroids, which split into smaller asteroids when hit, and enemies, which either chase or shoot at the player. The player loses a life when they are hit by an adversary, a bullet or an explosion. Destroying adversaries scores points and the best scores are kept in a file.

As a twist to the base version of Asteroids, we plan to add a kind of asteroid that explodes when shot by the player. This explosion grows over a short time and the player loses a life if they are inside of it. Asteroids arrive in waves that grow larger over time and a spawn method decides how regular and exploding asteroids are mixed within each wave.

2 Design

2.1 Data Structures

2.1.1 The Game State

The full state of our game is represented using a `GameState` data type. Each of its elements must exist.

```
data GameState = GameState
  { status       :: GameStatus
  , player       :: Player
  , playingField :: PlayingField  -- contains e.g. [Asteroid], [Enemy]
  , score        :: Int
  , highScores   :: [Int]         -- loaded from / saved to a file
  , wave         :: Int
  , rng          :: StdGen        -- random generator, threaded through
                                  -- the pure game logic
  }
```

The `GameStatus` data type holds status information about the game as a whole in relation to the user. Its state is thus driven by user interaction.

```
data GameStatus = Playing | Paused | GameOver
  deriving (Show, Eq)
```

2.1.2 The Initial State

The starting state of the game is defined in `Model.hs` by `initialState`, which is a pure function. The random generator is created and the high scores are loaded in `Main` (in `IO`) and passed in, so no `IO` is needed to build the first state. The assets are not part of the game state, as they are passed to `view` instead (see Section 2.3).

```
initialState :: StdGen -> [Int] -> GameState
initialState gen scores = GameState
  { status       = Paused
  , player       = initialPlayer
  , playingField = emptyField
  , score        = 0
  , highScores   = scores
  , wave         = 0
  , rng          = gen
  }

emptyField :: PlayingField
emptyField = PlayingField
  { asteroids  = []
  , enemies    = []
  , bullets    = []
  , explosions = []
  }
```

The playing field starts empty and `wave` starts at 0. The first frame's step therefore sees a cleared field and spawns wave 1 at random positions (see *Randomness* in Section 2.4). Spawning happens in one place only, so the initial state needs no special case for the first wave.

The player starts at the centre of the field, standing still and facing up, with 3 lives and no fire cooldown:

```
initialPlayer :: Player
initialPlayer = Player
  { playerPosition        = Position 0 0
  , playerVelocity        = Velocity 0 0
  , playerFacingDirection = 0             -- degrees, 0 = pointing up
  , playerLives           = startingLives
  , playerFireCooldown    = 0
  }
```

2.1.3 Position and Velocity

A `Position` represents the position of an object in the "world", not its rendered position on screen.

A `Velocity` represents the product of the movement direction of an object and its speed (in world units per second) in that direction. For example, an object with velocity $(1, 0)$ moves 1 world unit to the right per second.

The world unit is identical between `Position` and `Velocity`, and is thus standard within the entire physics layer of the game. By default, `playIO` assumes 1 world unit = 1 pixel. More on the rendering is described in Section 2.3. Objects may exist at world positions outside the display set for `playIO`.

```
data Position = Position
  { px :: Float  -- access using e.g. "px pos" where "pos :: Position"
  , py :: Float
  } deriving (Show, Eq)

data Velocity = Velocity
  { vx :: Float  -- access using e.g. "vx vel" where "vel :: Velocity"
  , vy :: Float
  } deriving (Show, Eq)
```

2.1.4 The Player

The `Player` is a required record field within `GameState`. In addition to a `Position` and `Velocity`, it also contains a `playerFacingDirection`, which is the angle of its body, where 0° means facing up.

```
data Player = Player
  { playerPosition        :: Position
  , playerVelocity        :: Velocity
  , playerFacingDirection :: Float  -- facing direction, separate from
    velocity
  , playerLives           :: Int
  , playerFireCooldown    :: Float  -- seconds until the player may shoot
     again
  } deriving (Show, Eq)
```

2.1.5 The Playing Field

The `PlayingField` is a required record field within `GameState`. It holds everything that exists in the world, besides the player.

```haskell
data PlayingField = PlayingField
  { asteroids  :: [Asteroid]
  , enemies    :: [Enemy]
  , bullets    :: [Bullet]
  , explosions :: [Explosion]  -- short-lived animations (animation
    requirement)
  } deriving (Show, Eq)
```

2.1.6 Adversaries, Explosions and Bullets

There are multiple kinds of adversaries in this game; currently `Asteroid` and `Enemy`. Asteroids, enemies and bullets each have a `Position`, a `Velocity`, and an integer size. Each unit of size represents 8 pixels, as defined by `sizeUnit` in `View.hs`:

```haskell
sizeUnit :: Float
sizeUnit = 8
```

Asteroid An asteroid moves in a single direction. It splits into 2 smaller asteroids when hit, which then move in different directions. It also contains an `AsteroidKind` tag, to represent different kinds of asteroids with different behaviour.

```haskell
data Asteroid = Asteroid
  { asteroidPosition :: Position
  , asteroidVelocity :: Velocity
  , asteroidSize     :: Int
  , asteroidKind     :: AsteroidKind
  } deriving (Show, Eq)
```

The `AsteroidKind` data type functions as a tag for the `Asteroid` data type and carries additional fields depending on the kind. For example, the `Exploding` constructor takes a `Float` argument, representing an explosion radius.

```haskell
data AsteroidKind
  = Regular
  | Exploding Float  -- explosion radius, from centre, not from edge
  deriving (Show, Eq)
```

Pattern matching on `AsteroidKind` then enables alternative behaviour for different kinds, using the additional fields. An exploding asteroid, in addition to splitting, spawns an `Explosion` at its current position when hit by a bullet or when colliding with the player.

```haskell
-- An explosion animation: it grows/fades with its age and is
-- removed after a fixed duration.
data Explosion = Explosion
  { explosionPosition :: Position
  , explosionAge      :: Float  -- seconds since it started
  , explosionRadius   :: Float
  } deriving (Show, Eq)
```

The collidable area (`currentRadius`) of the explosion is derived dynamically from its `explosionAge`, `explosionDuration` and `explosionRadius`. `explosionDuration` is a constant set in `Config.hs` for all explosions.

```haskell
explosionProgress :: Explosion -> Float  -- explosionAge /
    explosionDuration
currentRadius     :: Explosion -> Float  -- explosionProgress *
    explosionRadius
```

Enemy In our game, enemies are adversaries with more complicated movement patterns and no splitting behaviour. Enemies do not have a facing direction like the player does. This is derived dynamically and used immediately to face the player when a bullet is fired. It may be added if some reaction time is preferred between user movement and enemy turning.

```haskell
data Enemy = Enemy
  { enemyPosition  :: Position
  , enemyVelocity  :: Velocity
  , enemyBehaviour :: EnemyBehaviour
  } deriving (Show, Eq)
```

The movement pattern of an enemy depends on its `EnemyBehaviour` tag:

- A `Chaser` tries to slam into the player. Its velocity is changed to face the player, maintaining speed (the magnitude of the vector).
- A `Shooter` keeps firing bullets in the direction of the player, at a specified interval. Its velocity is changed to face the player, maintaining speed. If the shooter is within a specified distance of the player, its velocity is set to 0 to stop it from moving further.

```haskell
data EnemyBehaviour
  = Chaser         -- flies towards the player
  | Shooter Float  -- keeps its distance and shoots; seconds until next
    shot
  deriving (Show, Eq)
```

Bullet A bullet is a projectile fired by an enemy or by the player. Its velocity is set to face the player if shot by an enemy, or to the player's `playerFacingDirection` (multiplied by some speed) when shot by the player. A bullet additionally has an `Owner` field, used on collision between a bullet and an adversary or the player to check whether the bullet should deal damage to the other object.

```
data Owner = FromPlayer | FromEnemy
  deriving (Show, Eq)

data Bullet = Bullet
  { bulletPosition :: Position
  , bulletVelocity :: Velocity
  , bulletOwner    :: Owner
  -- , damagePoints :: Int
  } deriving (Show, Eq)
```

2.1.7 Collision Detection

All objects are treated as circles for collision detection. Two objects collide when the distance between their centres is smaller than the sum of their radii. What every collidable object has in common is captured in a type class:

```
class Collidable a where
  collisionPosition :: a -> Position
  collisionRadius   :: a -> Float

collides :: (Collidable a, Collidable b) => a -> b -> Bool
collides a b = distance (collisionPosition a) (collisionPosition b)
             < collisionRadius a + collisionRadius b
```

The radius of each object can be calculated from the dimensions of its image. Splitting an asteroid uses randomness, so collision handling takes the generator from the game state and stores the new one back:

```
handleCollisions :: GameState -> GameState
```

2.2 Interface and Input

Gloss provides a very simple way of capturing inputs and handling them. In `Controller.hs` we import `Event` from `Graphics.Gloss.Interface.IO.Game` and use pattern matching to handle different types of inputs. For example, `EventMotion` is used to capture the current cursor position.

Because we currently have no camera movement and 1 world unit equates to 1 pixel, the world coordinate system is equal to the screen coordinate system. Thus no special transformations need to be made between the mouse coordinates (in screen space) and player coordinates (in world space).

```haskell
input :: Event -> GameState -> IO GameState
input (EventMotion (x, y)) gs = do
  let p     = player gs
      pos   = playerPosition p
      dx    = x - px pos
      dy    = y - py pos
      angle = atan2 dx dy * 180 / pi  -- -180 to 180 degrees
  return gs { player = p { playerFacingDirection = angle } }
-- input (EventKey (Char 'w') Down _ _) gs = ...  -- move player up
input _ gs = return gs
```

2.3 Rendering

Rendering is done in `View.hs` and is pure. The function `view` is passed to `playIO` and only wraps the pure `render` function. `render` combines one picture per entity, using a separate drawing function per type:

```haskell
drawPlayer    :: Assets -> Player   -> Picture
drawAsteroid  :: Assets -> Asteroid -> Picture
drawEnemy     :: Assets -> Enemy    -> Picture
drawBullet    :: Assets -> Bullet   -> Picture
drawExplosion :: Explosion -> Picture
```

Each entity is drawn around the origin and moved to its position with `atPosition`, which uses `translate`. One world unit is one pixel and $(0, 0)$ is the centre of the screen.

We use bitmaps for the player, asteroids, enemies and bullets. Explosions are represented by orange discs that grow and fade out over their lifetime, using a simple Gloss circle.

Gloss draws pictures on top of each other in the order they are given, so `render` combines them in a fixed order: asteroids, enemies and bullets first, then explosions, the player, and interface text (such as the current score) last. This way explosions appear on top of the asteroids they come from, and the player is always visible.

The player, enemies, asteroids and bullets all have separate bitmaps. Because these are identical for every object of a type, they do not need to be stored within each instance. Instead, the `Picture` for each type is stored in an `Assets` record and taken from there when rendered:

```haskell
data Assets = Assets
  { playerAsset   :: Picture
  , asteroidAsset :: Picture
  , enemyAsset    :: Picture
  , bulletAsset   :: Picture
  }
```

The assets are then filled using `loadAssets`:

```haskell
loadAssets :: IO Assets
loadAssets = do
  playerBmp   <- loadBMP "playerimage.bmp"
  asteroidBmp <- loadBMP "asteroidimage.bmp"
  enemyBmp    <- loadBMP "enemyimage.bmp"
  bulletBmp   <- loadBMP "bulletimage.bmp"
  return Assets
    { playerAsset   = playerBmp
    , asteroidAsset = asteroidBmp
    , enemyAsset    = enemyBmp
    , bulletAsset   = bulletBmp
    }
```

In `Main.hs`, the assets are loaded with `loadAssets` and passed to `view`. Partially applying `view assets` gives the `GameState -> IO Picture` function that `playIO` expects.

```haskell
main :: IO ()
main = do
  gen    <- newStdGen
  scores <- loadHighScores highScoreFile
  assets <- loadAssets
  playIO (InWindow "Asteroids" (screenWidth, screenHeight) (0, 0))
         black                      -- background colour
         60                         -- frames per second
         (initialState gen scores) -- initial state
         (view assets)              -- view function
         input                      -- event function
         step                       -- step function
```

In `View.hs`, the assets are passed on to the individual draw functions:

```haskell
-- Entry point for Gloss: the rendering itself is pure, only the type is
    IO.
view :: Assets -> GameState -> IO Picture
view assets = return . render assets

-- Drawn back to front: later pictures appear on top.
render :: Assets -> GameState -> Picture
render assets gs = pictures $
     map (drawAsteroid assets) (asteroids field)
  ++ map (drawEnemy assets)    (enemies field)
  ++ map (drawBullet assets)   (bullets field)
  ++ map drawExplosion         (explosions field)
  ++ [drawPlayer assets (player gs), drawHud gs]
  where
    field = playingField gs

drawHud :: GameState -> Picture  -- score, lives, wave and pause/game-
    over text

-- Gloss rotates clockwise in degrees; 0 means pointing up.
drawPlayer :: Assets -> Player -> Picture
drawPlayer as p =
    atPosition (playerPosition p)
  . rotate (playerFacingDirection p)
  $ playerAsset as

drawEnemy :: Assets -> Enemy -> Picture
drawEnemy as e =
    ...
    $ enemyAsset as
drawAsteroid ...
drawBullet ...
```

2.4 Implementation of the Minimum Requirements

Player The player controls one spaceship using the keyboard, which can move around the rendered part of the playing field. The player's movement and facing direction are independent. The velocity is controlled with the keyboard (W, A, S, D). The facing direction (`playerFacingDirection`, an angle in degrees where 0 means up) always points towards the mouse cursor (see *Mouse input* in Section 2.5). Bullets are fired in the facing direction.

Pressing a movement key accelerates the player in that direction, and a constant drag slows the player down when no key is held. The player may fire bullets on input (e.g. pressing the spacebar) to destroy adversaries.

Enemies The game has antagonistic characters: asteroids and enemies (and, indirectly, asteroid explosions and enemy bullets). Asteroids and enemies vary greatly in behaviour. Adversaries try to win the game by lowering the player's `playerLives` to 0:

- by making direct contact with the player;
- by shooting bullets towards the player, which may damage the player on collision;
- by the player colliding with an explosion after an exploding asteroid explodes.

Over the course of the game, the difficulty increases by increasing the number of adversaries spawned. Asteroids may also spawn as part of an asteroid wave, with a given `SpawnMethod` dictating how the `AsteroidKind` of each asteroid is selected.

```
-- How many of each AsteroidKind we want to include in the wave
data AsteroidKindCount = AsteroidKindCount
  { countedKind   :: AsteroidKind
  , asteroidCount :: Int
  }

data SpawnMethod
  = Random
  | Alternating
  | Custom ([AsteroidKindCount] -> Rand [Asteroid])
    -- any function that takes the counts for each kind
    -- and creates a list of asteroids

makeAsteroidWave :: SpawnMethod -> Position -> [AsteroidKindCount]
                 -> Rand [Asteroid]
makeAsteroidWave (Custom customMethod) _ = customMethod
-- makeAsteroidWave Random      = ...
-- makeAsteroidWave Alternating = ...
```

Movement To allow objects to move, they must have an instance of the `Moveable` type class:

```
class Moveable a where
  getPosition :: a -> Position
  setPosition :: Position -> a -> a
  getVelocity :: a -> Velocity
  setVelocity :: Velocity -> a -> a
```

`Moveable` instances are made for `Player`, `Asteroid`, `Enemy` and `Bullet`:

```
instance Moveable Player where
  getPosition       = playerPosition
  setPosition pos p = p { playerPosition = pos }
  getVelocity       = playerVelocity
  setVelocity vel p = p { playerVelocity = vel }

instance Moveable Asteroid where ...
instance Moveable Enemy    where ...
instance Moveable Bullet   where ...
```

The `move` function takes a `Moveable` object and returns it with its position updated according to its velocity and the elapsed time `dt`:

```
move :: Moveable a => Float -> a -> a
move dt obj = setPosition newPos obj
  where
    currPos = getPosition obj
    currVel = getVelocity obj
    -- add (velocity * elapsed time) to the x and y of the position
    newPos  = Position (px currPos + vx currVel * dt)
                      (py currPos + vy currVel * dt)
```

Randomness All randomness comes from a single random generator stored in the game state (the `rng :: StdGen` field of `GameState`). The generator is created once in `Main` with `newStdGen`, which is the only impure step. From there it is threaded through the pure game logic. Every random function takes a generator and returns its result together with the updated generator. This pattern is captured in a type synonym in `Randomness.hs`, together with a helper that applies a random function to every element of a list:

```
type Rand a = StdGen -> (a, StdGen)

mapRand :: (a -> Rand b) -> [a] -> Rand [b]
```

Random objects are created in `Spawning.hs`. New asteroids appear at a random position just outside the edge of the screen, at least a minimum distance away from the player. They fly roughly towards the centre of the screen at a random speed, so that they always become visible:

```
randomSpawnPosition  :: Position -> Rand Position
randomInwardVelocity :: (Float, Float) -> Position -> Rand Velocity
spawnAsteroid        :: Position -> AsteroidKind -> Rand Asteroid
```

An entire wave is created with `makeAsteroidWave`, which first decides the order of the asteroid kinds according to the `SpawnMethod` (possibly at random), and then spawns each asteroid using the functions above.

In `Controller.hs`, `spawnWaveIfCleared` starts the next wave once the field is empty. It takes `rng` from the game state, uses it to create the wave, and stores the new generator back. Later waves contain more asteroids.

Besides spawn positions and velocities, we also randomize:

- the directions of the pieces of a split asteroid (`splitAsteroid :: Asteroid -> Rand [Asteroid]`);
- the order of asteroid kinds in a wave;
- the explosion radius of exploding asteroids;

- whether a new enemy is a `Chaser` or a `Shooter`, and a shooter's initial shot time (`spawnEnemy :: Position -> Rand Enemy`).

Animation When an exploding asteroid is hit by a player's bullet or collides with the player, it creates an `Explosion`, whose size changes over its lifetime. Explosions are stored in the `explosions` field of the playing field, separate from the asteroids. Its `explosionRadius` is the final radius, copied from the asteroid's kind. How long an explosion lasts is a constant in `Config.hs`:

```
explosionDuration :: Float
explosionDuration = 0.6
```

When an asteroid is hit by a player bullet or collides with the player, the collision code asks whether it should explode. Only the `Exploding` kind does, so this is decided by pattern matching on `AsteroidKind`:

```
explosionFor :: Asteroid -> Maybe Explosion
explosionFor a = case asteroidKind a of
  Exploding r -> Just (Explosion (asteroidPosition a) 0 r)
  Regular     -> Nothing
```

The explosion radius itself is chosen randomly when the asteroid spawns. The explosion copies it, so creating an explosion needs no randomness. Every frame, the pure `update` function ages all explosions by `dt` and removes the ones that are finished:

```
updateExplosions :: Float -> [Explosion] -> [Explosion]
updateExplosions dt = filter (not . isFinished) . map (ageBy dt)

ageBy      :: Float -> Explosion -> Explosion
isFinished :: Explosion -> Bool  -- explosionAge >= explosionDuration
```

Because the animation is driven by elapsed time and not by a frame counter, its speed does not depend on the frame rate. Because `step` does not call `update` while the game is paused, explosions also freeze during a pause without any extra code.

The appearance of an explosion is computed from its age alone, so no extra animation state is stored. Its progress runs from 0 to 1. The radius grows with the progress, while the colour fades out:

```
explosionProgress :: Explosion -> Float    -- explosionAge /
    explosionDuration
currentRadius     :: Explosion -> Float    -- explosionProgress *
    explosionRadius
drawExplosion     :: Explosion -> Picture  -- a solid circle, alpha = 1 -
     progress
```

`render` draws the explosions after the asteroids so that they appear on top. An explosion also causes the player to lose a life if they are within its

`currentRadius`. The same function is used for drawing and for collision, so what the player sees matches what can hit them.

Pause As part of the `GameState`, a `GameStatus` is stored (see Section 2.1.1). The game starts in the `Paused` state, so the player can get ready before the first wave appears. Pressing `p` toggles between `Playing` and `Paused`. We do not use Esc, because Gloss closes the window when Esc is pressed. Pressing `p` has no effect in the `GameOver` state.

```
togglePause :: GameStatus -> GameStatus
togglePause Playing  = Paused
togglePause Paused   = Playing
togglePause GameOver = GameOver
```

In `Controller.hs`, the `step` function used by Gloss to advance the world by one frame only calls the pure `update` function when the status is `Playing`. When the game is `Paused` or `GameOver`, the state is returned unchanged, so the world is frozen. The full definition of `step` is given under *IO* below, because it also saves the high scores.

IO The game keeps a list of high scores in a plain text file, with one score per line, sorted from highest to lowest. The file name and the number of scores kept are constants in `Config.hs`:

```
highScoreFile :: FilePath
highScoreFile = "highscores.txt"

maxHighScores :: Int
maxHighScores = 10
```

The file is read once in `Main` before `playIO` is called. The scores are then passed to `initialState`, which stays pure.

The file is written when the game ends. The pure `update` function decides when the game is over: it sets `status` to `GameOver` and inserts the final score into `highScores`. Only the actual writing happens in `IO`, which compares the status before and after the update:

```
step :: Float -> GameState -> IO GameState
step dt gs = case status gs of
  Playing -> do
    let gs' = update dt gs
    when (status gs' == GameOver) $
      saveHighScores highScoreFile (highScores gs')
    return gs'
  _ -> return gs
```

Everything except the file access itself is pure, so it can be tested without touching the file system:

```haskell
parseHighScores :: String -> [Int]          -- text to scores, ignores
    invalid lines
showHighScores  :: [Int] -> String          -- scores to text, one per
    line
insertScore     :: Int -> [Int] -> [Int]    -- adds a score, keeps the
    best maxHighScores
```

All high-score functions are defined in the module `HighScores.hs`. The pure functions (`parseHighScores`, `showHighScores`, `insertScore`) and the impure ones (`loadHighScores`, `saveHighScores`) are kept next to each other, but only the latter two use `IO`. They are the only functions in the module that touch the file system:

```haskell
loadHighScores :: FilePath -> IO [Int]
loadHighScores path = do
  exists <- doesFileExist path
  if exists
    then parseHighScores <$> readFileStrict path
    else return []  -- first run: no file yet

saveHighScores :: FilePath -> [Int] -> IO ()
saveHighScores path = writeFile path . showHighScores
```

2.5 Implementation of the Optional Requirements

Mouse input The facing direction is decoupled from the velocity vector, and is instead set to the direction from the player position to the mouse cursor. Mouse movement is captured using the `EventMotion` event from `Graphics.Gloss.Interface.IO.Game`, as shown by `input` in Section 2.2. Because there is no camera movement and 1 world unit equates to 1 pixel, no transformation between screen space and world space is needed.

Different enemies The game has 2 substantially different adversary types: asteroids and enemies. Enemies have substantially more complex movement mechanics and intelligence. Chasers chase the player. Shooters also approach the player, but maintain a minimum distance and shoot bullets at them. Asteroids, on the other hand, split when hit, and exploding asteroids may explode.

2.6 Separation of Pure and Impure Parts

We aim to keep the `IO` layer as thin as possible. The functions `view`, `input` and `step` all wrap a pure function that does the actual work. As such, `IO` is used for three things only:

- creating the random generator;

- loading the bitmaps;
- reading and writing the high-score file.

This allows us to keep the game logic, rendering and randomness pure.

Main The function `main` (see Section 2.3) performs all impure setup before the game starts. It creates the random generator, loads the high scores and loads the bitmaps. The generator and the scores are passed to `initialState`, which stays pure, and the assets are passed to `view`.

View As described in Section 2.3, the function `view` only lifts the pure `render` into `IO`:

```
view :: Assets -> GameState -> IO Picture
view assets = return . render assets
```

Input The function `input` (see Section 2.2) performs only pure operations and wraps the new `GameState` in `IO`. It is effectively pure because the fields of the new `GameState` are determined entirely by the previous `GameState`, constant fields and the given input. The same previous state and input always produce the same result.

Step The function `step` wraps the pure `update` function. Its only impure part is saving the high scores when the game has just ended. The pure `update` decides that the game is over and inserts the new score; then `step` performs the write (see the definition of `step` in Section 2.4). The write happens only once: after the first frame in `GameOver`, `step` no longer calls `update`.

Randomness We avoid `IO` by storing the generator in `GameState` as `rng` and threading it through the pure logic, using the type `Rand a` from `Randomness.hs`. Functions such as `spawnAsteroid`, `splitAsteroid` and `makeAsteroidWave` take a generator and return a new one. They are therefore pure functions: given the same generator, they always produce the same result.

High scores The file handling is split in the same way and all conversions are pure. Only `loadHighScores` and `saveHighScores` interact with the file and live in `IO`.

2.7 Abstraction

The `Moveable` type class described in Section 2.4 is our main abstraction over objects that move. Other abstraction ideas are:

- **Spawnable**: to allow spawning objects with a single `spawn` method, or spawning objects more easily as part of waves, for example `spawnWave :: Spawnable a => [a] -> GameState -> GameState`.
- **Explodeable**: if we want to extend explosion behaviour from exploding asteroids to other adversaries.
- **Drawable**: currently we have a separate draw function for the player, asteroids, enemies and bullets. There is a clear benefit in keeping them separate, however, as they each use different colours and Gloss shapes. Merging them would require pattern matching in `draw`, and overall the code would likely become less readable.